Q1. Below is a behavioral VHDL specification for the Stein's algorithm for computing the GCD of two non-negative integers.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity GCD_Calculator is
    Port (CLK, RESET, START, : in STD_LOGIC;
          X_IN : in UNSIGNED(31 downto 0);  -- Input number 1
          Y_IN : in UNSIGNED(31 downto 0);  -- Input number 2
          GCD_OUT : out UNSIGNED(31 downto 0);  -- Output GCD
          DONE : out STD_LOGIC); -- HIGH when GCD calculation is complete
end GCD_Calculator;

architecture Behavioral of GCD_Calculator is
    type State_Type is (IDLE, INIT, CALCULATE, FINISH);
    signal current_state, next_state: State_Type;
    signal x, y, x_temp, y_temp : UNSIGNED(31 downto 0);
    signal shift_count : NATURAL;
begin

    -- State transition process
    process(CLK, RESET)
    begin
        if RESET = '1' then
            current_state <= IDLE;
        elsif rising_edge(CLK) then
            current_state <= next_state;
        end if;
    end process;

    -- Control logic and algorithm implementation
    process(CLK)
    begin
        case current_state is
            when IDLE =>
                if START = '1' then
                    x <= X_IN;
                    y <= Y_IN;
                    shift_count <= 0;
                    next_state <= INIT;
                else
                    next_state <= IDLE;
                end if;
                DONE <= '0';

            when INIT =>
                -- Adjust inputs if either is zero
                if x = 0 then
                    GCD_OUT <= y;
                    next_state <= FINISH;
                elsif y = 0 then
                    GCD_OUT <= x;
                    next_state <= FINISH;
                else
                    next_state <= CALCULATE;
                end if;

            when CALCULATE =>
```

```
                -- Stein's algorithm using bitwise operations
                if x = y then
                    GCD_OUT <= x after 1 * shift_count;  -- Adjust by the common factors of 2
                    next_state <= FINISH;
                elsif x(0) = '0' then  -- Even number
                    x <= x srl 1;  -- Divide by 2
                elsif y(0) = '0' then
                    y <= y srl 1;
                elsif x > y then
                    x_temp <= (x - y) srl 1;
                    x <= x_temp;
                else
                    y_temp <= (y - x) srl 1;
                    y <= y_temp;
                end if;

            when FINISH =>
                DONE <= '1';  -- Signal that calculation is finished
                next_state <= IDLE;

            when others =>
                next_state <= IDLE;
        end case;
    end process;

end Behavioral;
```

a. Draw a flowchart for this algorithm.

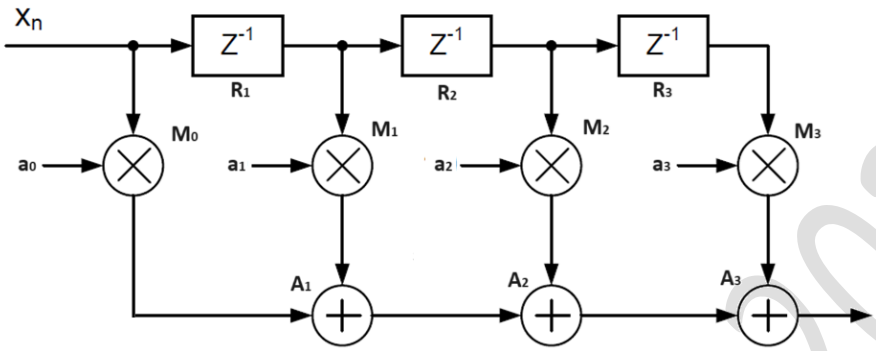
b. Considering the hardware implementation of this algorithm, draw a data flowgraph for this algorithm.

c. Draw the ASAP and ALAP scheduling diagrams for this algorithm.

d. Design a hardware for this algorithm with minimum hardware area (number of functional units)

e. Design a hardware for this algorithm with minimum latency (execution time).

Q2. Figure below shows a Finite Impulse Response (FIR) filter. The coefficients of the FIR filter (a_i) and the input signal (x_n) are constrained to have a signed fixed-point number format.



a. Assuming the following values/ranges for the input and the coefficients in this circuit, using interval analysis, find out the required bit-width for the functional units (adders (A_i) and multipliers (M_i)) and the memory units (Z^{-1} registers (R_i)).

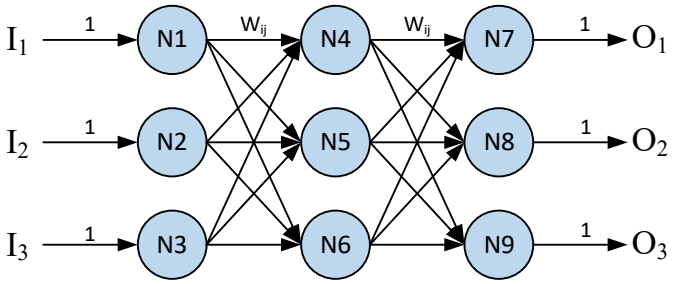
Variable/Constant	a_0	a_1	a_2	a_3	x_n
Range	[-50.01,100.01]	[-200.01,60.5]	[-50.01,130.1]	[-25.01,240.01]	[-100.01,100.01]

b. If we use a hardware implementation with 16-bit fixed-point (10.6) arithmetic, what would be the computational noise in the output.

c. Assuming the following coefficients, and input range of [-100.01,100.01] design a hardware implementation for this filter with minimum number of components and no computational error.

Constant	a_0	a_1	a_2	a_3
Value	24	65	63	48

Q3. Consider a simple neural network as shown in the figure below.



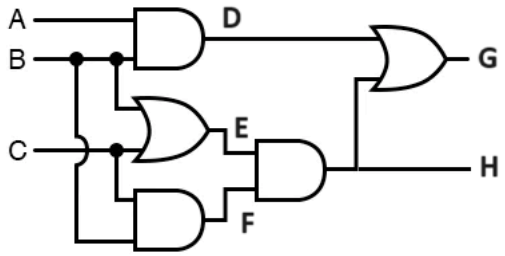
Synaptic weights in this network from neuron i to neuron j are W_{ij}. Neurons have a Heaviside step activation function $f(x)$, which is:

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

where x is the input to a neuron (this function is also known as a unit step function).

a) Draw the scheduling diagram for a single neuron (N4, for instance).
b) Using at most two adders and two multipliers and a single-bus approach, design a hardware implementation for a single neuron (N4, for instance) of this neural network.
c) Assuming T_{ADD}, T_{MUL}, T_{MEM}, to be delay of adders, multipliers, and memory units correspondingly, find the total delay for an inference using this network if we use your neuron design in part (b) and a flexible multi-bus implementation of the network. Draw a schematic of the hardware that you design for this network.

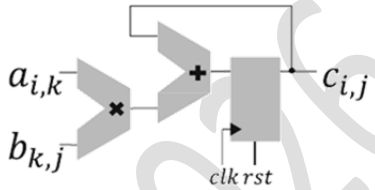
Q4. Figure below shows a logic gate level implementation of a single-bit Full-Adder.

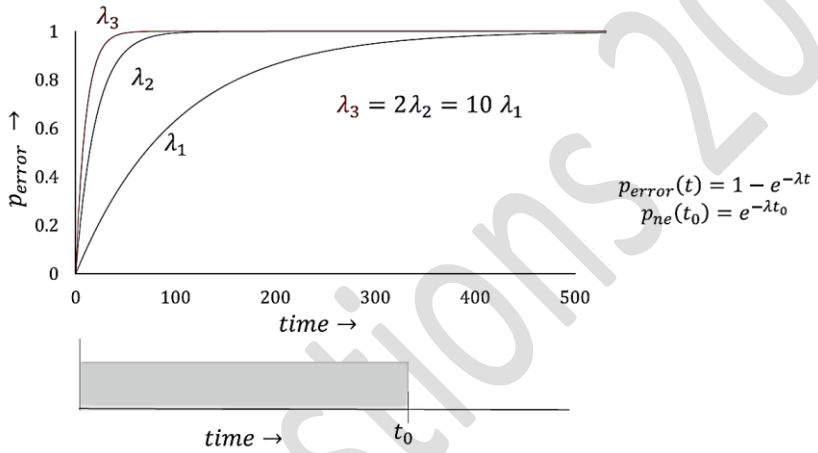


a) Using sensitive path algorithm suggests a minimum test pattern set for this circuit.
b) What are the Fault Cellarage and Test Coverage of your test pattern in %?
c) Discuss observability and testability of each point (A, B, C, D, E, F, G, H) in this circuit.

Q5. Figure below shows the matrix multiplication with the Multiply and Accumulate (MAC) considered as the computation module. Assuming the multiplication of two matrices, the MAC would compute dot products. We consider the computation of each element of the output matrix as a single operation. Also, for the error model, we assume an average fault rate of λ following a Poisson distribution. Correspondingly, the probability of an error occurring with increasing execution time is depicted in the second figure below.

$$\begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \\ b_{41} & b_{42} & b_{43} \end{bmatrix}$$





a) Design Dual Modular Redundancy (DMR) and Triple Modular Redundancy (TMR) redundant systems and calculate the probability of error in each case.

b) Using schematic diagram, redesign this system with N-Modular Redundancy (NMR), N-Temporal redundancy and with Information Redundancy. Draw a table and compare these three approaches in terms of different hardware costs such as area, time, and power consumption.

c) Discuss how someone can combine these methods to redesign this system with lower cost and higher reliability.